



Introduction to the LCS Problem

Intuition and Recursive Steps

Dr. Sumaiya Tabassum Nimi (STI)

Assistant Professor, Dept. of ECE — North South University

Understanding the Recursive Formula for the Longest Common Subsequence (LCS)

The Longest Common Subsequence (LCS) problem asks:

Given two strings X and Y , what is the longest sequence of characters that appears *in the same order* (but not necessarily contiguously) in both strings?

To solve this, we define $LCS(i, j)$ to be:

$$LCS(i, j) = \text{length of the LCS between } X[1..i] \text{ and } Y[1..j].$$

The recursive formula has two cases:

—

Case 1: Characters match

If the last characters of the prefixes match, i.e.,

$$X[i] = Y[j],$$

then this matching character *must* be part of the LCS of these prefixes. So we take it and reduce the problem to the remaining smaller prefixes:

$$LCS(i, j) = 1 + LCS(i - 1, j - 1).$$

Intuition:

- If $X[i]$ and $Y[j]$ match, it is like finding a puzzle piece that fits.
- Since both strings end with the same character, that character adds +1 to the LCS.
- The only task left is to find the LCS of the remaining strings:

$$X[1..i - 1] \quad \text{and} \quad Y[1..j - 1].$$

Example:

$$X = \text{"ABCD"} , \quad Y = \text{"ACD"}.$$

Since $D = D$:

$$LCS(4, 3) = 1 + LCS(3, 2).$$

Case 2: Characters do *not* match

If the last characters do not match, i.e.,

$$X[i] \neq Y[j],$$

we cannot include both characters in the LCS because they conflict. So we try two possibilities:

$LCS(i-1, j)$: ignore the last character of X

$LCS(i, j-1)$: ignore the last character of Y

Then take the better (larger) result:

$$LCS(i, j) = \max(LCS(i-1, j), LCS(i, j-1)).$$

Intuition:

- If the characters do not match, one of them must be “wrong” for the current alignment.
- So we explore both possibilities:
 - Maybe deleting $X[i]$ helps align things.
 - Maybe deleting $Y[j]$ helps instead.
- We choose whichever path yields the longer subsequence.

Example:

$$X = \text{"ABD"} , \quad Y = \text{"ACE"}.$$

Since $D \neq E$:

$$LCS(3, 3) = \max(LCS(2, 3), LCS(3, 2)).$$

Putting it all together

$$LCS(i, j) = \begin{cases} 0, & \text{if } i = 0 \text{ or } j = 0, \\ 1 + LCS(i - 1, j - 1), & \text{if } X[i] = Y[j], \\ \max\{LCS(i - 1, j), LCS(i, j - 1)\}, & \text{if } X[i] \neq Y[j]. \end{cases}$$

Intuition summary:

1. If characters match $\Rightarrow$ take them and shrink both strings.
2. If characters don't match $\Rightarrow$ try removing each character one at a time and pick the better result.
3. Base case: If either string is empty, $LCS = 0$.

This recursive structure reveals that LCS is built from smaller subproblems, making it ideal for dynamic programming.

Recursive Simulation of the LCS Problem

Given two strings:

$$X = \text{ABCB DAB}, \quad Y = \text{BDCAB}$$

We want to compute:

$$LCS(X, Y)$$

1. The Recursive Formula (Intuition)

Let $X[1..m]$ and $Y[1..n]$ be the prefixes of the two strings, and let the last characters be X_m and Y_n .

$$LCS(X[1..m], Y[1..n]) = \begin{cases} 0, & m = 0 \text{ or } n = 0 \\ 1 + LCS(X[1..m - 1], Y[1..n - 1]), & X_m = Y_n \\ \max(LCS(X[1..m - 1], Y[1..n]), LCS(X[1..m], Y[1..n - 1])), & X_m \neq Y_n \end{cases}$$

Intuition:

- If the last characters match, they *must* be part of the LCS.
- If they do not match, the LCS must come from either:
 1. dropping the last char of X , or
 2. dropping the last char of Y .

2. Start the Recursion

We compute $\text{LCS}(\text{ABCBDA}, \text{BDCAB})$.

Compare last characters:

$$X_7 = B, \quad Y_5 = B$$

They match!

$$\Rightarrow \text{LCS}(X, Y) = 1 + \text{LCS}(\text{ABC BDA}, \text{BDCA})$$

So we now compute:

$$\text{LCS}(\text{ABCBDA}, \text{BDCA})$$

3. Continue Recursing

Compare final characters:

$$A \text{ vs } A$$

Match!

$$\Rightarrow \text{LCS} = 1 + \text{LCS}(\text{ABCB}, \text{BDC})$$

Now compute:

$$\text{LCS}(\text{ABCB}, \text{BDC})$$

4. Compare last characters: D vs C

$$X_5 = D, \quad Y_3 = C$$

No match $\rightarrow$ take max of two branches:

$$\text{LCS}(\text{ABCB}, \text{BDC}) = \max \left(\text{LCS}(\text{ABCB}, \text{BDC}), \text{LCS}(\text{ABCB}, \text{BD}) \right)$$

Evaluate each branch.

5. Branch 1: $\text{LCS}(\text{ABCB}, \text{BDC})$

Compare last characters:

$$B \text{ vs } C$$

No match:

$$\text{LCS}(\text{ABCB}, \text{BDC}) = \max \left(\text{LCS}(\text{ABC}, \text{BDC}), \text{LCS}(\text{ABCB}, \text{BD}) \right)$$

5.1 Sub-branch A: LCS(ABC, BDC)

Compare last characters:

C vs C (match)

$$= 1 + \text{LCS}(\text{AB}, \text{BD})$$

Now compute LCS(AB, BD):

Compare last chars:

B vs D (no match)

$$\text{LCS}(\text{AB}, \text{BD}) = \max(\text{LCS}(\text{A}, \text{BD}), \text{LCS}(\text{AB}, \text{B}))$$

Compute each:

- LCS(A, BD): Compare A vs D $\rightarrow$ no match

$$= \max(\text{LCS}(\text{A}, \text{B}), \text{LCS}(\text{A}, \text{BD})) = \max(0, 0) = 0$$

- LCS(AB, B): B matches B

$$= 1 + \text{LCS}(\text{A}, \text{B}) = 1$$

So:

$$\text{LCS}(\text{AB}, \text{BD}) = 1$$

Thus:

$$\text{LCS}(\text{ABC}, \text{BDC}) = 1 + 1 = 2$$

5.2 Sub-branch B: LCS(ABCB, BD)

Compare last characters:

B vs D (no match)

$$= \max(\text{LCS}(\text{ABC}, \text{BD}), \text{LCS}(\text{ABCB}, \text{B}))$$

We already know:

$$\text{LCS}(\text{ABC}, \text{BD}) = 1$$

Now compute:

$$\text{LCS}(\text{ABCB}, \text{B})$$

Last characters:

B vs B (match)

$$= 1 + \text{LCS}(\text{ABC}, \text{B}) = 1$$

Thus:

$$\text{LCS}(\text{ABCB}, \text{BD}) = \max(1, 1) = 1$$

So for Branch 1:

$$\text{LCS}(\text{ABCB}, \text{BDC}) = \max(2, 1) = 2$$

6. Branch 2: LCS(AB CBD, BD)

Compare last characters:

$$D \text{ vs } D \text{ (match)}$$

$$= 1 + \text{LCS}(\text{ABCB}, \text{B})$$

We computed earlier:

$$\text{LCS}(\text{ABCB}, \text{B}) = 1$$

So:

$$\text{LCS}(\text{AB CBD}, \text{BD}) = 1 + 1 = 2$$

7. Combine the two branches

$$\text{LCS}(\text{AB CBD}, \text{BDC}) = \max(2, 2) = 2$$

8. Go back to earlier matches

Recall:

$$\text{LCS}(\text{ABC BDAB}, \text{BDCAB}) = 1 + 1 + \text{LCS}(\text{AB CBD}, \text{BDC})$$

Because we had two earlier character matches: B–B and A–A.

Thus:

$$\text{LCS} = 2 + 2 = 4$$

9. Final Answer

$$\boxed{\text{LCS}(\text{ABC BDAB}, \text{BDCAB}) = 4}$$

One possible LCS is:

$$\boxed{\text{BCAB}}$$

Other valid LCS sequences include:

$$\text{BDAB}, \text{BCDB}, \text{BCAB}$$

Understanding the Longest Common Subsequence (LCS) Recurrence

1. Example Strings

$$X = \text{ABCB DAB}$$

$$Y = \text{BDCAB}$$

We compute $LCS(7, 5)$ because $|X| = 7$ and $|Y| = 5$.

2. Recursive Expansion of $LCS(7, 5)$

We evaluate:

$$LCS(7, 5) \quad \text{where } X[7] = B, Y[5] = B$$

Since the characters match:

$$LCS(7, 5) = 1 + LCS(6, 4)$$

Now expand $LCS(6, 4)$ where $X[6] = A, Y[4] = A$:

$$LCS(6, 4) = 1 + LCS(5, 3)$$

Next expand $LCS(5, 3)$ where $X[5] = D, Y[3] = C$ (no match):

$$LCS(5, 3) = \max\{LCS(4, 3), LCS(5, 2)\}$$

Continue expanding:

$$LCS(4, 3) : X[4] = B, Y[3] = C \Rightarrow (\text{no match})$$

$$LCS(4, 3) = \max\{LCS(3, 3), LCS(4, 2)\}$$

$$LCS(3, 3) : X[3] = C, Y[3] = C \Rightarrow (\text{match})$$

$$LCS(3, 3) = 1 + LCS(2, 2)$$

$$LCS(2, 2) : X[2] = B, Y[2] = D \Rightarrow (\text{no match})$$

$$LCS(2, 2) = \max\{LCS(1, 2), LCS(2, 1)\}$$

Eventually $LCS(1, 2), LCS(2, 1)$, etc. break down to base cases when either index reaches 0.

This continues until all recursive branches terminate.

3. Overlapping Subproblems

During the full expansion, many subproblems appear repeatedly. Below is a representative list.

$LCS(4, 3)$ appears multiple times

$LCS(3, 3)$ appears multiple times

$LCS(2, 2)$ appears multiple times

$LCS(1, 2)$ appears multiple times

$LCS(5, 2)$ appears multiple times

Why does this happen? Because when characters do not match, the recurrence creates both:

$$LCS(i - 1, j) \quad \text{and} \quad LCS(i, j - 1)$$

These expansions often lead to the *same pairs* (i, j) later.

4. Counting Overlaps (Conceptual)

Below is an approximate count for this example (counts vary slightly depending on recursion order):

$LCS(4, 3)$: appears about 4 times

$LCS(3, 3)$: appears about 3 times

$LCS(2, 2)$: appears about 4–6 times

$LCS(1, 2)$: appears about 5 times

$LCS(5, 2)$: appears about 3 times

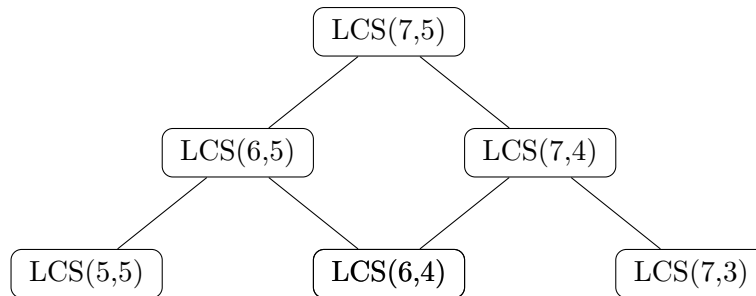


Figure 1: Overlapping subproblem: $LCS(6,4)$ appears twice (and many more times deeper in the tree).